

Task 4 — Multi-Agent Orchestration and Trajectory Evaluation

TRACK Agentic AI — Intern Programme, Task 4 of 5

PACKAGE	packages/orchestrator
PREREQUISITES	Task 3 (reuses its tool surface and trajectory format). TypeScript.
COST	Zero. Everything runs locally.

Why this task exists

A single agent with a good loop still has one blind spot: it grades its own work. Real systems split responsibilities — one process proposes, another checks — because a model reviewing its own output inherits its own blind spots. The moment you add a second agent, you inherit an entire new failure category that single-agent evals don't catch: agents can agree with each other, defer to each other, or loop between each other, and the final answer can look fine while the process that produced it was nonsense.

So Task 4 is not “add more agents for more capability.” It's: build the smallest multi-agent system that has a real disagreement built into its design, and then prove your evaluation can tell a good disagreement from a rubber stamp.

Setup

Same runtime as Task 3 (Node 20+, TypeScript strict, Vitest, Ollama qwen2.5:7b-instruct), plus a message bus — in-process is fine, no need for real queues — that passes typed messages, not raw strings.

Explicitly forbidden

LangGraph, CrewAI, AutoGen, or any pre-built multi-agent framework. You may read their source. You may not import them.

What you are building

A three-role system — Planner, Executor, Critic — that takes a small feature request against the same repo from Task 3 (e.g. “add a rate limiter to this endpoint”) and produces a patch plus a record of the Critic's review.

```
pnpm orch run --task "add rate limiting to /api/login"
pnpm orch eval
pnpm orch eval --compare baseline.json
```

Requirements

1. Typed message protocol

Every inter-agent message is a typed object (Plan, Patch, Review, RevisionRequest), never a raw string blob one agent parses out of another's prose. Define these types before writing any agent logic — this is the “public

interfaces first" part of your design doc made literal.

```
interface Review {  
  verdict: 'approve' | 'revise';  
  issues: { severity: 'blocker' | 'minor'; note: string; location?: string }[];  
  turnsUsed: number;  
}
```

2. A Critic that can actually block

The Critic is a separate model call with a separate prompt and, critically, no access to the Executor's chain of reasoning — only the final patch and the original task. It must be able to send work back with a `RevisionRequest`, and the Executor must address the specific issues raised, not just resubmit with cosmetic changes. Cap revision rounds (e.g. 3) with a defined outcome when the cap is hit: escalate, don't loop forever.

3. Shared state, not shared context

Planner, Executor, and Critic do not share one giant context window. Each gets exactly the state it needs for its role, explicitly assembled by the harness. This is deliberate: it's the difference between "multi-agent" and "one long prompt with role-play labels," and it's also what makes each agent's failure mode isolatable when you evaluate it.

4. Trajectory-level evaluation, not just outcome

This is the actual point of the task. For each golden task, score two things separately:

- Outcome: did the final patch solve the task (tests pass, requirement met)?
- Process: was the path there sane? Did the Critic ever approve a patch with an unaddressed blocker (rubber-stamping)? Did the Executor address the Critic's actual issues, or just resubmit similar code (ignored feedback)? Were there redundant round-trips that added no information?

A system can score well on outcome and badly on process. That gap is what single-shot evals never show you, and it's the reason trajectory evaluation is now standard practice for anything more complex than a single tool call.

5. Golden evaluation set — 15 tasks

CATEGORY	COUNT	DEFINITION
Well-specified	10	Small feature requests against the shared repo, with acceptance criteria you write down in advance
Seeded flaw	3	The Executor's first attempt has a real, injectable flaw you plant — an off-by-one, a missing await — specifically to test whether the Critic catches it
Ambiguous	2	The correct Planner behavior is to ask a clarifying question rather than guess; decide in advance what a reasonable clarifying question looks like so you can grade it

6. Metrics

METRIC	WHAT IT MEASURES
task success rate	Outcome quality
critic catch-rate	Of the seeded flaws, the fraction flagged as a blocker on the first review
rubber-stamp rate	Approvals a human reviewer would not have approved
mean revision rounds	Per task
redundant round-trip ratio	Round-trips that added no new information
total tokens / wall-clock	Per task, all agents combined

Deliverable: RESULTS.md

A table comparing: single-agent baseline (Executor only, no Critic) vs + Critic with no revision cap vs + revision cap vs + isolated context (no shared chain-of-reasoning). Then attribute: if the Critic without a revision cap doesn't change catch-rate but does blow up token cost, that's a real finding — say so.

Hard parts you should expect

- Sycophantic critics. A Critic prompted without enough independence will approve almost anything the Executor produces, especially if the prompt frames the Executor as competent. Design against this — don't just hope the model is honest.
- Confabulated blockers. The opposite failure — a Critic that invents issues to seem thorough. Both failure modes look like “the Critic is working” from the outside; only trajectory-level scoring against your seeded flaws distinguishes them.
- Context leakage. It's easy to accidentally let the Critic see the Executor's own justification for its code, which lets the Critic rubber-stamp the reasoning instead of judging the artifact. Keep the boundary real.
- Escalation with no owner. When the revision cap is hit, decide explicitly what happens — surface the full disagreement to a human reviewer, don't silently pick the last patch produced.

Deliverables

- packages/orchestrator — Planner, Executor, Critic, and the message bus
- evals/golden-orchestrator.jsonl — 10 well-specified + 3 seeded-flaw + 2 ambiguous tasks
- Metrics harness reporting all six metrics
- RESULTS.md — the comparison table plus the attribution paragraph
- NOTES.md — sycophancy vs over-critical failure modes, and what you did about each
- DESIGN.md — written before the code

Design doc, before any code

One page, reviewed and approved before you write a line:

- Public interfaces and types
- The three most likely failure modes, and your plan for each
- What you are deliberately not building, and why
- Open questions

Review gate

Your reviewer will verify these by hand before merge.

- Feed the Executor a patch with a seeded blocker and confirm the Critic's review is logged as 'revise' with the correct issue located — not just a generic “looks fine.”
- Force the revision cap by having the Executor resubmit an unaddressed patch 3 times; confirm the system escalates rather than looping a 4th time.
- Read three full trajectories cold, without looking at the outcome label, and independently judge process quality; compare your read to the automated process score.
- Every number in RESULTS.md reproduces from a fresh clone.